

# PARAKH: AI-Assisted Tender Evaluation

## Hackathon Prototype Guide & Summary

### 1. Core Philosophy: Accountability, Not Automation

*"The system's job is not to make eligibility decisions. Its job is to make decisions transparent, consistent, and defensible so that officers can explain them to anyone who asks."*

Government procurement (like for the CRPF) is high-stakes. The real problem isn't just "slow reading"—it's the **Explanation Gap**. Officers fear making decisions they can't defend to auditors (CAG), vigilance teams, or RTI inquiries. PARAKH acts as a sovereign assistant, ensuring every AI suggestion is backed by proof.

### 2. The Four Subsystems (The Brain of PARAKH)

**Tender Understanding Engine (TUE):** Reads the tender documents (NIT, QR) to extract rules and create a structured **Criteria Manifest** (the blueprint).

**Bidder Document Intelligence (BDI):** Handles messy bidder files (scans, photos, handwriting). It converts them into structured fields like turnover and certificate validity using OCR.

**Evaluation & Ambiguity-Resolution Engine (EARE):** The logic engine. It maps evidence to criteria and uses a **Deterministic Three-State Verdict**: Eligible, Not Eligible, or Needs Manual Review.

**Human-in-the-Loop Workbench (HLWAS):** The officer's dashboard. It shows AI suggestions, allows overrides with **Reason Codes**, and generates an immutable audit trail.

### 3. Key Technical Concepts

---

- **Three-State Model:** Unlike "black-box" AI, PARAKH never guesses. If confidence is low or a rule is subjective, it defaults to **Manual Review**.
- **Audit Trail & Hash Chaining:** Every action is saved as an event. Using SHA-256 chaining, the system ensures that logs are tamper-evident and 100% auditable.
- **Hard vs. Graduated Criteria:** **Hard** rules are numeric (e.g., Turnover > 50 Cr). **Graduated** rules are subjective (e.g., "similar work") and always require human approval.
- **Confidence Scoring:** AI provides a score (0.0 to 1.0) for every extraction. If the score is below a threshold (e.g., 0.75), it triggers a flag.

### 4. Hackathon Tech Stack (Recommended)

---

- Image Processing

| Layer                    | Technology Choice                       |
|--------------------------|-----------------------------------------|
| Frontend                 | Next.js / React with TypeScript         |
| Backend API              | FastAPI (Python)                        |
| Database                 | MongoDB (for flexible document storage) |
| OCR / PDF Parsing        | PyMuPDF, PaddleOCR, pdfplumber          |
| OpenCV (for enhancement) |                                         |

### 5. Step-by-Step Hackathon Roadmap

---

1. **Finalize Use Case:** Pick one category (e.g., "Riot Gear") and create 1 sample tender and 3 mock bidders.
2. **Build TUE:** Create the pipeline to turn a tender PDF into a **JSON Criteria Manifest**.
3. **Build BDI:** Implement OCR to extract key fields (PAN, GSTIN, Turnover) from bidder documents.
4. **Logic Engine (EARE):** Implement the rule engine to compare extracted values against the manifest.
5. **The Workbench:** Build the "Tri-pane" dashboard (Criterion Context, Evidence Viewer, Decision Tools).

6. **Audit & Export:** Generate a final PDF/Excel report showing exactly why a bidder was passed or failed.